

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №2

Тема «Переміщення, обертання та масштабування»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 19:

1. Змініть колір фону та моделі.
2. Змініть фігуру на будь-яку іншу.
3. Виконайте декілька різних трансформацій з фігурою.
4. Наведіть коментарі до коду.
5. Внесіть індивідуальні зміни до коду.

РЕЗУЛЬТАТ

```
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

// Розміри вікна
const GLint WIDTH = 800, HEIGHT = 800;
// Конвертація градусів в радіани
const float toRadians = 3.14159265f / 180.0f;
// Кут для обертання
float curAngle = 0.0f;
// Ідентифікатори OpenGL об'єктів
GLuint VAO, VBO, shader, uniformModel;
// Змінні для анімації руху (трансляції)
bool direction = true;
float triOffset = 0.0f;
float triMaxOffset = 0.5f; // Зменшено максимальний зсув
float triIncrement = 0.001f; // Збільшено швидкість руху
// Змінні для анімації розміру (масштабування)
bool sizeDirection = true;
float curSize = 0.4f;
float maxSize = 0.9f; // Змінено максимальний розмір
float minSize = 0.2f; // Змінено мінімальний розмір

// Змінні для додаткової анімації: пульсація кольору
float colorR = 1.0f;
float colorG = 0.0f;
float colorB = 0.0f;
float colorIncrement = 0.005f;
```

```

// **Вершинний шейдер**
// Перетворює 3D координати вершини в 2D координати на екрані.
static const char *vShader = "
\n #version 330
\n\

\n\
// 'pos' - вхідна змінна для координат вершини (location = 0 відповідає
glVertexAttribPointer)
layout(location = 0) in vec3 pos;
\n\
\n\
// 'model' - уніформ змінна для матриці моделі (трансляція, обертання,
масштабування)
uniform mat4 model;
\n\
\n\
void main()
\n
{
    \n
        // Фінальна позиція вершини: застосовуємо матрицю моделі до
вхідної позиції
        gl_Position = model * vec4(pos, 1.0);
    \n
}
";

// **Фрагментний шейдер**
// Визначає фінальний колір кожного пікселя (фрагмента).
static const char *fShader = "
\n #version 330
\n\

\n\
// 'colour' - вихідна змінна, фінальний колір фрагмента
out vec4 colour;
\n\
\n\
// Нова уніформ змінна для динамічного кольору
uniform vec4 ourColor;
\n\

\n\
void main()
\n
{
    \n
        // Присвоюємо колір, який буде оновлюватись в main циклі
colour = ourColor;
    \n
}
";

// Ідентифікатор уніформ змінної кольору у фрагментному шейдері
GLuint uniformOurColor;

// 2. Змініть фігуру на будь-яку іншу (Квадрат/Прямокутник)

```

```

void CreateSquare()
{
    // Координати вершин квадрата (два трикутники)
    // Зверніть увагу, що координати знаходяться у діапазоні [-1, 1]
    GLfloat vertices[] = {
        // Перший трикутник
        -0.5f, -0.5f, 0.0f, // (0) Лівий нижній
        0.5f, -0.5f, 0.0f, // (1) Правий нижній
        0.5f, 0.5f, 0.0f, // (2) Правий верхній
        // Другий трикутник
        -0.5f, -0.5f, 0.0f, // (0) Лівий нижній
        0.5f, 0.5f, 0.0f, // (2) Правий верхній
        -0.5f, 0.5f, 0.0f // (3) Лівий верхній
    };

    // 1. Створення масиву вершин (Vertex Array Object - VAO)
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    // 2. Створення буфера вершин (Vertex Buffer Object - VBO)
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    // Завантаження даних вершин у VBO
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
        GL_STATIC_DRAW);

    // 3. Налаштування вказівників атрибутів вершин
    // layout(location = 0) in vec3 pos;
    // 0: індекс атрибута
    // 3: кількість компонент на вершину (x, y, z)
    // GL_FLOAT: тип даних
    // GL_FALSE: чи нормалізувати дані
    // 0: крок між наборами атрибутів (0 = щільно упаковано)
    // 0: зсув початку даних
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
    glEnableVertexAttribArray(0);

    // 4. Відв'язка VBO та VAO
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

// Додавання та компіляція окремого шейдера
void AddShader(GLuint theProgram, const char *shaderCode, GLenum
shaderType)
{
    GLuint theShader = glCreateShader(shaderType);
    const GLchar *theCode[1];
    theCode[0] = shaderCode;
    GLint codeLength[1];
    codeLength[0] = strlen(shaderCode);

    glShaderSource(theShader, 1, theCode, codeLength);
    glCompileShader(theShader);

    // Перевірка на помилки компіляції
    GLint result = 0;
    GLchar eLog[1024] = {0};

```

```

    glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(theShader, 1024, NULL, eLog);
        fprintf(stderr, "Error compiling the %d shader: '%s'\n",
shaderType, eLog);
        return;
    }

    glAttachShader(theProgram, theShader);
}

// Компіляція та лінкування шейдерної програми
void CompileShaders()
{
    shader = glCreateProgram();
    if (!shader)
    {
        printf("Failed to create shader\n");
        return;
    }

    AddShader(shader, vShader, GL_VERTEX_SHADER);
    AddShader(shader, fShader, GL_FRAGMENT_SHADER);

    GLint result = 0;
    GLchar eLog[1024] = {0};

    // Лінкування програми
    glLinkProgram(shader);
    glGetProgramiv(shader, GL_LINK_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error linking program: '%s'\n", eLog);
        return;
    }

    // Валідація програми
    glValidateProgram(shader);
    glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error validating program: '%s'\n", eLog);
        return;
    }

    // Отримання location для уніформ змінних
    uniformModel = glGetUniformLocation(shader, "model");
    uniformOurColor = glGetUniformLocation(shader, "ourColor"); //
Отримання location для нового кольору
}

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())

```

```

{
    printf("GLFW failed to start :(");
    glfwTerminate();
    return 1;
}

// Налаштування версії OpenGL (Core Profile 3.3)
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

// Створення вікна
GLFWwindow *mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Custom
OpenGL Animation", NULL, NULL);
if (!mainWindow)
{
    printf("GLFW window creation failed :(");
    glfwTerminate();
    return 1;
}

int bufferWidth, bufferHeight;
glfwGetFramebufferSize(mainWindow, &bufferWidth, &bufferHeight);

// Встановлення контексту для поточного вікна
glfwMakeContextCurrent(mainWindow);

// Ініціалізація GLEW
glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK)
{
    printf("GLEW failed to start >_<");
    glfwDestroyWindow(mainWindow);
    glfwTerminate();
    return 1;
}

// Встановлення області перегляду (Viewport)
glViewport(0, 0, bufferWidth, bufferHeight);

// Створення фігури (Квадрата)
CreateSquare();
// Компіляція та лінкування шейдерів
CompileShaders();

// **Головний цикл рендерингу**
while (!glfwWindowShouldClose(mainWindow))
{
    // Обробка подій (введення з клавіатури/миші)
    glfwPollEvents();

    // **3. Виконайте декілька різних трансформацій з фігурою.**

    // Анімація 1: Рух по діагоналі (Трансляція)
    if (direction)
    {
        triOffset += triIncrement;
    }
}

```

```

    }
    else
    {
        triOffset -= triIncrement;
    }
    if (abs(triOffset) >= triMaxOffset)
    {
        direction = !direction;
    }

    // Анімація 2: Обертання
    curAngle += 0.5f; // Збільшено швидкість обертання
    if (curAngle >= 360)
    {
        curAngle -= 360;
    }

    // Анімація 3: Пульсація розміру (Масштабування)
    if (sizeDirection)
    {
        curSize += 0.001f; // Збільшено швидкість пульсації
    }
    else
    {
        curSize -= 0.001f;
    }
    if (curSize >= maxSize || curSize <= minSize)
    {
        sizeDirection = !sizeDirection;
    }

    // **5. Внесіть індивідуальні зміни до коду. (Пульсація
кольору)**
    // Анімація 4: Пульсація кольору (з червоного до синього)
    if (colorR > 0.0f && colorB <= 0.0f) {
        colorR -= colorIncrement;
        colorB += colorIncrement;
    } else if (colorB > 0.0f && colorR <= 0.0f) {
        colorB -= colorIncrement;
        colorR += colorIncrement;
    }

    // 1. Змініть колір фону
    // Очищення кольорового буфера (темно-синій/фіолетовий)
    glClearColor(0.1f, 0.0f, 0.2f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Використання шейдерної програми
    glUseProgram(shader);

    // Ініціалізація матриці моделі (одинична матриця)
    glm::mat4 model(1.0f);

    // Застосування трансформацій у зворотному порядку (Scale ->
Rotate -> Translate)

    // 1. Трансляція (Рух)

```

```

        // model = model * T (де T - матриця трансляції)
        model = glm::translate(model, glm::vec3(triOffset, triOffset,
0.0f));

        // 2. Обертання навколо осі Z (в площині екрана)
        model = glm::rotate(model, curAngle * toRadians,
glm::vec3(0.0f, 0.0f, 1.0f));

        // 3. Масштабування
        // Оскільки ми малюємо 2D, компонент Z не використовується, але
його необхідно задати, щоб уникнути помилок.
        model = glm::scale(model, glm::vec3(curSize, curSize,
1.0f)); // Змінено Z на 1.0f

        // Надсилання матриці моделі в вершинний шейдер
        // uniformModel: location уніформ змінної
        // 1: кількість матриць
        // GL_FALSE: не транспонувати матрицю
        // glm::value_ptr(model): вказівник на дані матриці
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));

        // Надсилання динамічного кольору у фрагментний шейдер
        glUniform4f(uniformOurColor, colorR, colorG, colorB, 1.0f);

        // Малювання фігури
        glBindVertexArray(VAO);
        // Малюємо 6 вершин (два трикутники) для квадрата
        glDrawArrays(GL_TRIANGLES, 0, 6);
        glBindVertexArray(0);

        // Зупинка використання шейдерної програми
        glUseProgram(0);

        // Обмін буферами (виведення відмальованого зображення на
екран)
        glfwSwapBuffers(mainWindow);
    }

    // Очищення ресурсів GLFW
    glfwTerminate();
    return 0;
}

```